

CPSC 8740 Final Project Report: ETAGlance

Rahul Chamarthi

1 Introduction and Overview:

1.1 Motivation and Problem Statement

The concept for **ETAGlance** has been in my head for quite some time, born out of a specific personal necessity. As a Graduate Student who is simultaneously working full-time as a consultant, my schedule is chaotic to put it lightly. Getting class or catching the bus to the main campus on time can be difficult. My days often involve getting as much sleep as I can and immediately loading maps on my phone upon waking up to see how long the commute to the bus stop is.

While modern iOS maps applications are powerful, they have a specific problem; they do not natively allow users to “pin” live Estimated Time of Arrival (ETA) feeds for specific locations directly to the Lock Screen or Home Screen for glanceable consumption. To check an ETA, a user typically must unlock their device, open a specific maps application, and input the destination or select it from their pinned locations.

1.2 Proposed Solution

To address this gap, I developed **ETAGlance**, a sleek, minimalist iOS application designed with a “widget-first” philosophy. The primary objective of this project was to build an application that allows users to track ETAs to their favorite locations without needing to open a maps application repeatedly.

The core value proposition of ETAGlance relies on integration of iOS widgets. Satisfaction of the project’s core objective depended on successfully creating a widget that could be pinned to the User’s Home Screen, providing real-time travel data at a glance.

1.3 Project Scope and Development Timeline

The project was executed over a structured 4-week timeline, moving from initial architecture to a stable MVP.

- **Week 1 (11/5 – 11/11) – Foundations:** Defined problem scope and utilized AI tools (Stitch) for UI exploration. Technical deliverables included initializing the Xcode project, creating the FavoriteLocation data model and scaffolding the HomeView with mock location cards.
- **Week 2 (11/12 – 11/18) – Interactivity & State:** Enabled user functionality by building the “Add Favorite Location” flow with multi-model transport selection. Implemented JSON persistence to save and load locations locally and added swipe-to-delete interactions.

- **Week 3 (11/19 – 11/25) – Location, MapKit and Widget:** Integrated MapKit to build the ETA engine and created the WidgetKit extension for Home Screen updates. Implemented LocationService and RoutingService to handle real-world GPS data.
- **Week 4 (11/26 – 12/5) – Stability, Testing and MVP:** Completed the “Edit Favorite” UI flow and polished the Widget layout. I also planned on creating comprehensive unit tests, however, illness during this week forced me to pivot to manual testing to ensure the MVP was stable.

2 AI Tool Selection Methodology:

2.1 Strategy and Selection Criteria

With a strict four-week timeline and limited bandwidth due to my full-time job, my strategy was to prioritize development speed. I didn’t just want to use AI for the sake of it; I needed the tools to handle the heavy lifting of implementing the base code and initial UI creation. I evaluated tools based on two simple criteria: Context Awareness(can the tool understand my codebase?) and Rapid Prototyping(can it generate usable assets quickly?). After testing several options, I narrowed my tool stack down to Cursor for development and Stich for asset creation.

2.2 Coding and Development: Cursor

For the development environment, I chose Cursor over standard Xcode autocompletion. The main reason was its ability to maintain context and its role as the current industry standard. Cursor could iterate over my entire codebase and understand exactly where to insert new code segments when prompted with high level design-based prompts. However, the tool wasn’t perfect. It frequently hallucinated deprecated Swift syntax, requiring me to manually intervene by cross referencing Apple’s official documentation to resolve compilation errors.

2.3 UI Design: Stich

For the interface I used Stich to generate high-fidelity wireframes and assets. I selected this tool because of its low learning curve compared to Figma. The streamlined user interface allowed me to rapidly translate text prompts into visual components that looked like native iOS elements. There was definitely wasted time spent during this project trying to iterate through Figma’s AI features.

2.4 Environment and Libraries: Xcode & CoreLocation

While AI handled development speed, I had no choice but to use Xcode for the build system. In-built simulation that allowed me to ensure native performance was a big plus. Libraries like CoreLocation and WidgetKit were non-negotiable for the GPS and Home Screen features I needed to meet my design vision. The biggest learning curve came with configuring “App Groups” This feature is required to allow the Widget and App to share the same database allowing for the real time ETA refresh function.

3 User Interface Design and Integration:

3.1 Design Workflow: From Wireframe to Prototype

The design philosophy for ETAGlance was centered on “information at a glance”. I utilized Stich to rapidly generate UI layouts that emphasized this core principle. The layouts generated in the wireframes emphasized a clearly defined navigation flow and screen hierarchy.

As the design evolved into high-fidelity prototypes there was an emphasis on trying to adhere to modern iOS design patterns. I prioritized card-based layouts for clear separation of data, standard SF symbols for intuitive icons, and segmented controls for easy selection. In the end this process yielded a complete application lifecycle, mapping the user flow from the Home dashboard (HomeView) to location creation (AddFavoriteView) and finally to the widget interface (ETAWidget)

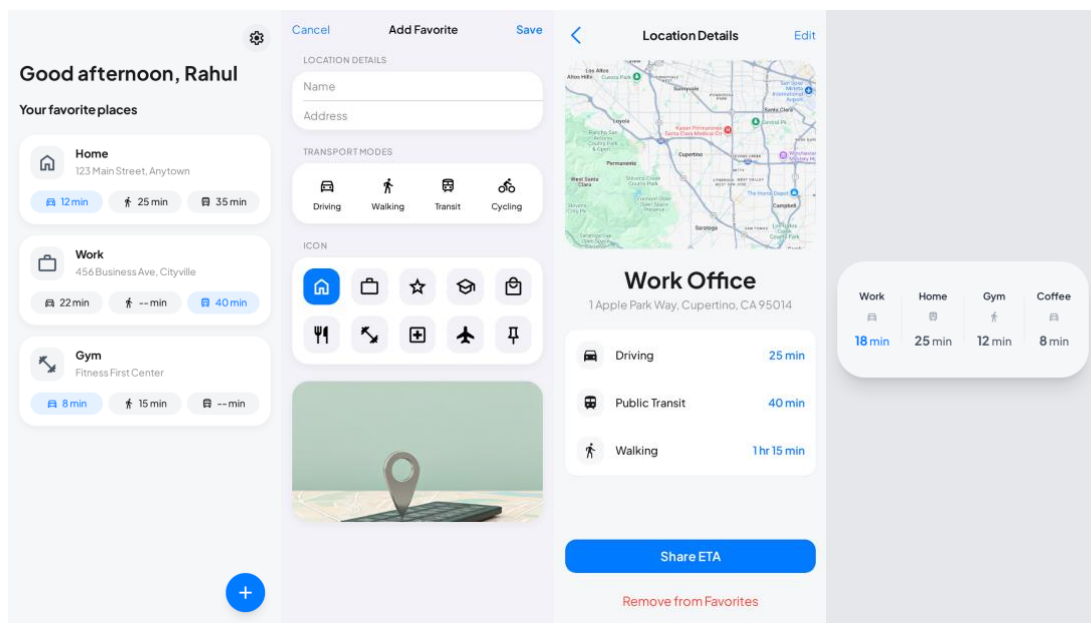


Image 1. Initial Wireframe Results

3.2 Technical Integration and Architecture

Following the design phase, the focus shifted to converting these visual wireframes into a real application. I utilized Cursor to pass the wireframes in and generate live SwiftUI code that not only matched them but connected them to the data layer using `@StateObject` and `@EnvironmentObject` wrappers.

One of the biggest obstacles that was encountered while doing this leg work, was ensuring consistency of theme and information across the application and the widget. Several iterations were required to make sure that Cursor got small details such as spacing, colors and corner radii correct. I then integrated the Home, Add, and Edit views directly with the persistence layer so that any changes, such as new locations, would immediately be reflected across the UI.

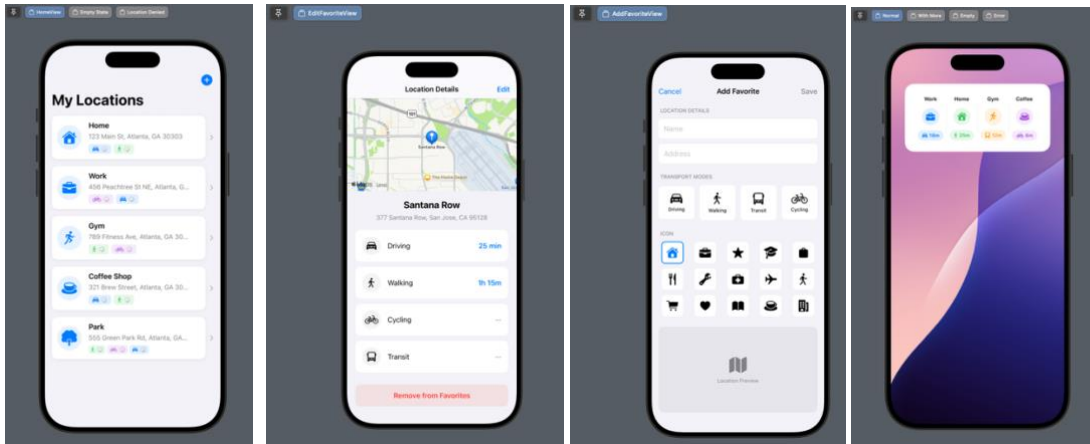


Image 2. Results of translating Wireframe to Xcode

4 Testing and Refinement:

4.1 Usability Testing Methodology

Given the visual and human-centered success metrics for this application, the testing strategy relied heavily on iterative manual validation rather than automated scripts. I performed several tests going throughout the entire app flow to make sure changes progressed as expected. The location widget was designed as a 1x4 grid, basically a row of 4 locations. However, actions such as deleting multiple locations needed to reflect in the widget. When fewer than 4 locations were present, the widget had to maintain consistent spacing, and when more than 4 locations were present a signifier had to be added.

For the sake of brevity, I will not list every such iteration I made to catch edge cases that needed an appropriate reflection in the UI. But needless to say, with the help of informal peer feedback from my software engineering colleagues several LLM tokens were burned.

4.2 Bug Fixing and Iterative Refinements

The discussion from the previous section leads us into this one. The testing phase uncovered several critical issues that required immediate refinement. I talked a bit about the requirement to have multiple views for the widgets previously, but another challenge was the visual alignment of ETA data. Initial builds showed that variable string lengths (e.g. “55 min” vs. “2hr 45min”) caused the UI cards to shift unpredictably. To resolve this, I implemented fixed-width containers and standardized font scaling to ensure consistent alignment regardless of data length.

Functionally, the most robust refinement was the implement of error-handling logic for the Routing Service. Real-world testing revealed that MapKit requests would occasionally timeout or fail in areas with poor cellular coverage. I added a retry mechanism and a “stale data” policy for the widget. Making sure that users are presented with the last known good ETA or a clear “Unable to Load” state rather than a blank screen or crash.

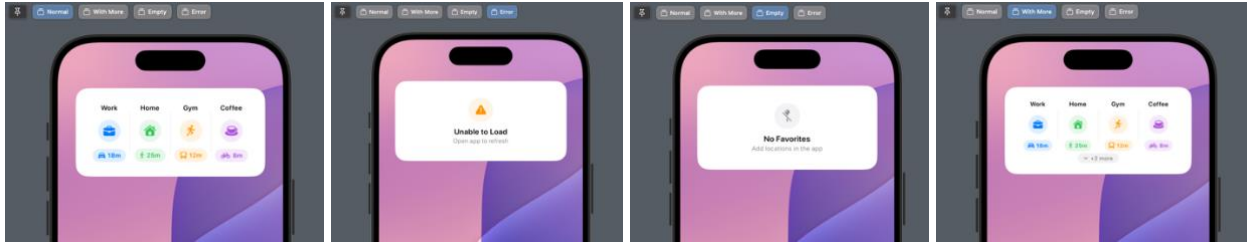


Image 3. Alternate Widget views created after iterative testing

4.3 Technical Challenges and Limitations

While my manual testing philosophy produced a stable MVP, I did run into specific constraints. A primary technical limitation discovered during testing was Apple MapKit's inconsistent support for public transit data in certain regions. To mitigate this, I implemented a fallback mechanism that flags unsupported transit routes to the user rather than failing silently.

A final point to note is that my original project plan included a comprehensive suite of unit tests for the persistence layer. However, due to an illness I contracted at the beginning of Week 4, I was unable to complete this specific objective.

5 Final Conclusions & Future Work:

5.1 Final Conclusions

The purpose of this project from a personal objective was to implement ETAGlance and all the objectives I have listed above, however, my secondary objective and main reason for taking this course was to become more intimately familiar with the tools I will have to use in my next job following Graduate School. To this end the project served as a practical case study in AI-assisted development. I was incredibly impressed with how within 4 weeks, with minimal knowledge of modern iOS development, I was able to develop a solid MVP.

It only reinforced the ideas I summarized in my reflection I submitted separately on Canvas. These tools are here to stay, and quite frankly I don't think they will replace us, but they are a massive productivity boost to those who embrace them. So many times, throughout undergrad I felt like I had great ideas I wanted to develop, however, the barrier or time investment required to learn new syntax made them non-viable. It really is a new age for all of us in this field, and I for one am thrilled by the possibilities.

5.2 Future Work

Moving forward, my objectives remain consistent with the project's inception. I believe in the practical utility of this application and aim to see it published on the App Store. To achieve this, I intend to implement a full unit test suite, User Profile support, and an enhanced routing engine featuring real-time traffic weighting. Additionally, I plan to integrate calendar-based predictive routing and perform final UI refinements.

6 Instructions to Run:

Given that the application is not currently on the App Store I recommend the following steps to run the ETAGlance project locally:

1. **Download the Source Code from:** <https://github.com/rchamarthi-edu/CPSC8740/tree/main/Final%20Project>
2. **Unzip the Source Code:** Download and extract the ETAGlance project folder from ETAGlance.zip
3. **Open in Xcode:** Double-click the ETAGlance.xcodeproj file inside the ETAGlance project folder to launch the project
4. **Run in Simulator:** Press Cmd + R to build and run
5. **Launch Application:** Once the app is installed, press Cmd + Shift + H to go to the Home Screen and see the Widget for the Application. If the widget is not present long press the home screen till the widgets giggle. Add the widget to the home screen by selecting edit in the top left and searching for “ETAGlance”. If the Widget is already present you may press it to launch into the Application’s Home Screen.